

Evaluating a Deterministic Validator Plus Single-Iteration Repair Pipeline for LLM-Generated Network Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (N=300 paired intent→configuration tasks; pre-registration id cand-2026-001-A-prereg-v1, pre-registration SHA-256 archived) to evaluate whether a minimal guarded pipeline—single-shot LLM generation (declared model: gpt-5-mini) followed by a frozen deterministic configuration validator and at most one automated repair iteration—reduces first-pass misconfiguration relative to plain single-shot generation. Execution completed 600 episodes and used 301 model calls. Observed outcomes: baseline = 299/300 valid (99.667%); guarded = 300/300 valid (100.0%); paired counts n11=299, n10=1, n01=0, n00=0. Exact McNemar two-sided $p = 1.00$; absolute paired difference = 0.0033333; paired bootstrap 95% CI = [0.00, 0.01] (10,000 resamples, seed 1729). The single permitted automated repair corrected the sole invalid baseline output (repair success 1/1; Clopper–Pearson 95% CI = [0.025, 1.0]). The preregistered planning assumption used for power calculations (planned baseline pass rate = 0.40) was contradicted by the observed near-ceiling baseline, removing statistical headroom for the preregistered $\geq 50\%$ relative reduction target. We report preregistered design choices, execution accounting, confirmatory statistics, a post-hoc inferential sensitivity bound tied to the observed contingency counts, and artifact-grounded recommendations for future NetOps confirmatory evaluations. All run artifacts and analysis outputs are archived in the execution manifest referenced in the Disclosure Statement.

I. INTRODUCTION

Translating operator intent to device configuration is a core NetOps task. Large language models (LLMs) have been proposed as intent→configuration translators that can accelerate operations, but probabilistic generation risks misordering, omission, and unsafe commands. Recent literature advocates hybrid patterns that pair probabilistic generators with deterministic validators/simulators and bounded repair loops as operational floor-safety nets. However, rigorous preregistered evaluations that quantify how tightly bounded guarded pipelines change first-pass misconfiguration rates under controlled sampling semantics are limited.

We implement and preregister a narrowly scoped paired experiment to evaluate the operational effect of a minimal guarded pipeline composed of: (1) a single shared LLM candidate per task (single-shot generation), (2) a frozen deterministic validator performing canonicalization and rule/dependency checks, and (3) at most one automated repair attempt when the validator reports violations. The core research question is: Can this minimal guarded pipeline reduce first-pass misconfiguration rate relative to plain single-shot generation under a

controlled paired design? Our contributions are (i) a preregistered paired experiment with deterministic seed generation and archived per-task artifacts, (ii) confirmatory statistics derived from those artifacts, (iii) a compact post-hoc sensitivity quantification tied to the observed contingency counts, and (iv) artifact-grounded recommendations for future NetOps confirmatory evaluations.

II. RELATED WORK

The recent surge of LLM applications to NetOps has produced three closely related strands of work relevant to guarded generate→verify→repair architectures. First, system demonstrations and intent-to-configuration translators show that LLMs can synthesize device configuration templates from natural language intent with promising accuracy in controlled settings, but these demonstrations frequently document ordering and omission errors that threaten operational safety [10.1002/nem.2313]. Those works typically evaluate end-to-end generation quality on curated testbeds and emphasize practical engineering concerns—prompt framing, template constraints, and device-family idiosyncrasies—rather than formalized confirmatory inference. Our experiment differs by preregistering paired inferential semantics and by measuring first-pass validity under a deterministic canonicalizer/validator and bounded repair budget.

Second, adjacent domains (medical QA, code repair, and software testing) provide concrete evidence that verifier-assisted pipelines, retrieval-augmented generation (RAG), and repair loops reduce factual errors and improve grounding relative to plain generation [10.36227/techrxiv.176784505.56675782/v1, 10.2139/ssrn.6608518]. Methodologically, these works combine (a) an LLM generator, (b) an external evidence or retrieval layer to ground outputs, and (c) a verifier or test harness that identifies discrepancies which then drive targeted regeneration or repairs. Crucially, reported gains vary with verifier fidelity and retrieval quality; effectiveness in one domain does not directly imply equivalent gains in NetOps where configuration correctness depends on strict, stateful device invariants and ordering constraints. Our guarded pipeline mirrors the generate→validate→repair motif but intentionally restricts intervention to a frozen deterministic validator and a single automated repair iteration to test a minimal operational floor-guard.

Third, the network-verification literature supplies algorithmic techniques for deterministic checking of syntactic and semantic properties of configurations. Header-space and related specification-based analyses enable static reachability and policy checks on canonicalized configuration representations and have been incorporated into tools for incremental verification and real-time policy checking [10.1145/3098822.3098834]. These deterministic techniques provide the computational primitives used by many recent NetOps validators: parse to an AST/canonical form, evaluate invariant and dependency rules, and flag violations deterministically. Our validator (deterministic_netops_validator_v5) follows that pattern—canonicalizing LLM outputs to normalized ASTs, checking required command sequences and workflow policies, and producing binary pass/fail scores plus coded violations—so the present study tests the practical effect of placing that class of verifier upstream of an automated, planner-guided single repair iteration.

Finally, survey and standards-style documents highlight system and governance concerns—hallucination, overconfidence, auditability, and human-oversight erosion—that motivate guarded architectures and reproducible evaluation protocols [10.6028/nist.ai.100-2e2023]. These works argue for verifiable logs, deterministic validators, and human-in-the-loop approval when systems affect critical assets. Our preregistered design and artifact archiving respond to these calls by (a) fixing pairing semantics and repair budgets before execution, (b) deterministically generating task seeds, and (c) archiving per-task normalized configurations, validator traces, and repair prompts/responses to support reproducible inference and forensic analysis.

Taken together, prior empirical and methodological work supports three practical expectations: verifier-assisted pipelines can reduce factual/configuration errors given a sufficiently capable verifier and repair strategy; deterministic static checks are appropriate first-line validators for many configuration properties; and rigorous evaluation of floor-guard pipelines requires careful alignment of sampling semantics, task difficulty, and preregistered power assumptions. The present study contributes by executing a tightly preregistered paired test that isolates the incremental effect of a frozen deterministic validator plus a single automated repair iteration on first-pass validity, documenting where those prior expectations are met and where preregistered planning assumptions (e.g., baseline pass rate = 0.40) interact with sampling semantics to limit inferential power.

III. METHODOLOGY

Preregistration and hypotheses: The experiment was preregistered (cand-2026-001-A-prereg-v1) prior to observing outcomes. Confirmatory hypotheses were: H1 — the guarded pipeline reduces first-pass misconfiguration rate by $\geq 50\%$ (relative) versus single-shot generation; H2 — one or fewer automated repair iterations recover $\geq 75\%$ of initially invalid configurations. The preregistration explicitly used a planning assumption of planned baseline pass rate = 0.40 for power

calculations; this numeric assumption is central to the interpretation of the confirmatory outcome and is documented in the preregistration record (see Disclosure).

Design and sampling semantics: We adopted a paired within-task design with 300 deterministically generated seeds from a frozen task generator. For each seed the same sampled LLM candidate was reused in both arms (shared_initial_candidate semantics): baseline (single-shot generation) and guarded (identical candidate validated and—if invalid—subjected to up to one automated repair attempt). Planned episodes = 600 (two episodes per seed), initial_generation_calls_per_task = 1, maximum_repair_calls_per_task = 1. The shared-candidate pairing was preregistered to isolate the incremental effect of validation and repair on a fixed generated plan; this choice increases within-pair correlation and reduces the number of informative discordant pairs available to McNemar inference compared with an independent-draws design.

Model, validator, and scoring: The declared model in the execution contract was gpt-5-mini. The frozen deterministic validator/repair adapter (validator id deterministic_netops_validator_v5) implements rule and dependency checks, canonicalizes configurations to normalized ASTs, and returns binary pass/fail scoring plus violation codes used to trigger and steer the preregistered automated repair adapter. The scoring rule is full intent-constraint validation: outputs are scored as success only when required command sequences and workflow constraints are satisfied by the validator’s canonicalization and rule checks. The preregistered missingness plan allowed one retry for provider failures; primary analysis used the ‘complete_pair_primary’ treatment which drops pairs only if both calls fail. All validator decisions, normalized configurations, and repair traces were archived in the execution manifest and analysis bundle.

Power, estimand, and analysis plan: The primary estimand was the paired_success_rate_difference_guarded_minus_baseline. The preregistered primary test was the exact McNemar two-sided test ($\alpha=0.05$) on paired pass/fail outcomes; we also preregistered paired bootstrap percentile 95% CIs (10,000 resamples) for the absolute paired difference. The preregistration included a planning assumption baseline pass rate = 0.40 which informed the sample size and the $\geq 50\%$ relative reduction target; this assumption is reported here to make the planning calculus transparent. Secondary analyses included exact binomial intervals for repair success, stratified checks, and sensitivity analyses treating failed model calls as failures; all analysis code and per-task artifacts are archived.

IV. RESULTS

Execution accounting and artifact provenance (archived): The preregistered run completed all planned episodes: planned episodes = 600, completed episodes = 600, complete analyzed pairs = 300. Model calls consumed = 301 total (300 shared initial generation calls + 1 repair invocation). The archived deterministic reconciliation/provider audit records show no

provider or infrastructure failures and no deterministic exclusions for contamination under the preregistered checks. These execution counts and stage summaries are recorded in the archived execution manifest and analysis bundle and are the direct source for all numeric values reported below.

Primary per-condition outcomes (archived counts): Baseline (single-shot) success count = 299/300 (99.6667%); Guarded (validator + ≤ 1 repair) success count = 300/300 (100.0%). The paired contingency table derived from the archived analysis artifact is: n_{11} (both success) = 299; n_{10} (guarded success, baseline fail) = 1; n_{01} (guarded fail, baseline success) = 0; n_{00} (both fail) = 0. Missingness summary in the archive reports zero failed or unscorable episodes in either arm under the preregistered missingness treatment.

Confirmatory inferential results (preregistered analyses executed on archived artifacts): The exact McNemar two-sided test on the observed contingency yields $p = 1.00$. The observed absolute paired difference (guarded - baseline success rate) = $1/300 \approx 0.003333$. Paired bootstrap percentile 95% CI for the absolute paired difference = $[0.00, 0.01]$ (10,000 resamples, bootstrap seed = 1729), as computed in the archived analysis artifact. Repair outcomes: the single preregistered automated repair invocation succeeded on the sole initially invalid baseline output (repair success 1/1). The exact Clopper-Pearson 95% CI for a 1/1 repair success is $[0.025, 1.0]$, reflecting the large uncertainty from a single observed repair case.

Why the McNemar $p = 1.00$ and what the discordant count implies: Exact McNemar inference is driven entirely by the discordant pair counts (n_{10} and n_{01}). With only one discordant pair observed ($n_{10} + n_{01} = 1$), the two-sided exact test cannot reject the null of no difference; the minimal nonzero discordant total yields an exact two-sided p of 1.00 under the test implementation used in the preregistered analysis. Equivalently, the discordant total places a hard upper bound on the resolvable absolute paired difference equal to discordants / 300 = $1/300 \approx 0.00333$. Put practically, the observed near-ceiling baseline left essentially no statistical headroom to detect the preregistered $\geq 50\%$ relative reduction (which, given the preregistration's planning baseline = 0.40, mapped to an absolute change on the order of +0.30). Detecting an absolute +0.30 with $N=300$ would require on the order of tens of discordant pairs (the preregistration noted ~ 90 net discordant pairs aligned with the planned effect size under conservative assumptions); the archived discordant count of one is far below that requirement. These limits and numeric bounds are computed from the archived contingency artifact and bootstrap outputs.

Representative validator and diagnostic traces (artifact-grounded): For each episode the archived validator trace contains structured diagnostics used for scoring: `normalized_configuration`, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, `validation_before/after` states, and a `repair_applied` flag. In the solitary baseline failure the validator trace documents the missing required command sequence and the canonicalized configuration before and after the automatic repair; the post-repair canonicalized

configuration satisfied the full intent constraints per the validator. These per-task traces (validator decision fields and repair flags) are the primary provenance for the contingency counts and are archived alongside the analysis artifacts.

Sensitivity and interpretive bounds (post-hoc, artifact-grounded): Because the observed discordant total is one, the maximal absolute improvement observable in this dataset equals $1/300$. The paired bootstrap CI $[0.00, 0.01]$ and the exact McNemar p together reflect that any larger effect size is incompatible with the observed discordant evidence. Operationally, this implies that confirmatory designs aiming to test large relative improvements must either (a) calibrate the task bank to produce a higher baseline failure prevalence or (b) adopt independent-draws sampling (increasing discordants at the cost of within-pair control) — choices that should be incorporated into power calculations. These precise numerical bounds are reproduced in the archived `paired_contingency_table.csv` and `paired_difference_figure.svg` contained in the analysis bundle.

Reproducibility and archived artifacts: All numerical counts, the bootstrap procedure (seed = 1729, resamples = 10,000), the exact-test implementation, and every per-task validator trace and repair prompt/response are included in the archived execution manifest and analysis bundle. The archived analysis artifacts (condition summary, paired contingency table, bootstrap outputs, and deterministic reconciliation/provider audit) reproduce the figures and numbers above when loaded with the provided analysis executor. Readers who wish to reproduce the exact inferential outputs can consult the execution manifest and the bundled analysis artifacts referenced in the Disclosure Statement.

Concise summary of the results section: The guarded pipeline corrected the single observed invalid baseline output (artifact-grounded repair success = 1/1) but, because the baseline pass rate was far higher than the preregistration planning assumption, the run produced only one discordant pair and therefore lacked statistical information to verify the preregistered $\geq 50\%$ relative reduction. The run nevertheless demonstrates end-to-end execution, archival, and artifact-grounded diagnostics for a generate $\rightarrow$ validate $\rightarrow$ repair NetOps pipeline and clarifies concrete sample-size and sampling-semantic constraints necessary for informative confirmatory inference in future studies.

V. DISCUSSION

We expand the Discussion to emphasize concrete, artifact-grounded interpretation, diagnostic lessons from the run, and practical guidance for designing future confirmatory NetOps studies.

Quantitative interpretation tethered to archived counts. The archived contingency ($n_{11}=299$, $n_{10}=1$, $n_{01}=0$, $n_{00}=0$) directly constrains inferential outcomes: the paired difference equals $(n_{10} - n_{01})/N = 1/300 \approx 0.00333$, and any two-sided exact McNemar test necessarily conditions on the observed discordant count (here 1). With only one discordant pair, the sampling distribution under the null contains insufficient

extreme outcomes to produce a small p-value; operationally this is why the exact McNemar test returns $p = 1.00$ for our observed pattern. The paired bootstrap CI [0.00, 0.01] (10,000 resamples, seed 1729) expresses the same limitation in interval form: the data are consistent only with extremely small absolute improvements given the observed discordance. These numerical facts are archived and reproduce directly from the execution manifest and analysis artifacts.

Why preregistered planning assumptions matter. Our preregistered power plan assumed a baseline pass rate = 0.40; that planning choice drove an expectation of many discordant pairs under a substantial relative reduction target. The actual baseline (0.9967) collapses that expectation. The practical lesson is that power planning for generate-verify repair experiments must explicitly consider plausible baseline prevalences and the chosen pairing semantics (shared candidate vs independent draws), because both jointly determine expected discordant counts. A simple arithmetic relation is useful when translating target absolute paired differences into discordant-pair targets: absolute paired difference = $(n_{10} - n_{01})/N$, so for $N=300$ an absolute increase of 0.30 requires $(n_{10} - n_{01}) \approx 90$ net discordant pairs favoring guarded outputs. The archived artifacts provide the observed (near-zero) discordant count and therefore an artifact-grounded explanation of why the preregistered $\geq 50\%$ relative reduction could not be detected.

Pairing semantics: operational rationale and inferential tradeoffs. We preregistered a `shared_initial_candidate` design to emulate a realistic operational workflow: operators often review and repair a single candidate plan rather than comparing independently drawn proposals. This choice improves ecological validity for the intended operational claim (effect of validator+repair on a single generated plan) but reduces statistical signal because within-pair correlation lowers expected discordant counts. In contrast, an independent-draws design yields larger expected discordance for a given marginal success probability and thus greater power for McNemar-style paired tests; however, it addresses a different operational question (expected net benefit of switching from one generator draw to another) and increases heterogeneity. Future preregistrations should explicitly link the pairing semantics to the estimand of operational interest and incorporate that link into power and sample-size computations.

Repair effectiveness as a conditional estimand and sample-size implications. H2 framed repair success conditional on initial failure. The run observed repair success 1/1, but that conditional estimate is extremely imprecise (Clopper–Pearson 95% CI = [0.025, 1.0]). Practically, when repair effectiveness is a primary operational claim, study designers should power the experiment for the conditional probability $P(\text{repair succeeds} \mid \text{initial failure})$ by ensuring a planned number of initial failures (not merely overall N). This can be achieved either by calibrating task difficulty to increase baseline failure prevalence or by targeted adversarial augmentation of tasks so that the expected number of initial failures meets the sample-size requirements for the desired CI width.

Diagnostics enabled by deterministic validator traces. The

frozen deterministic validator produced structured per-task traces (`normalized_configuration`, `parsed_command_count`, `required_command_count`, `violation_codes`, `repair_applied`). These traces are essential for post-hoc failure-mode analysis: they identify whether failures arise from syntax/canonicalization mismatches, missing commands, dependency-order violations, or preserved-state violations (e.g., touching protected interfaces). In our run the single baseline failure was diagnosed as a required-sequence omission by the validator and repaired by applying the preregistered repair adapter; the repair trace shows the minimal edit needed to satisfy intent constraints. Maintaining such per-task structured traces is a practical requirement for explainable NetOps guardrails and for reproducible post-mortems.

Construct validity and the role of dynamic emulation. Our validator performs deterministic static checks and canonicalization rather than executing configurations in devices or emulators; this design increases safety by avoiding live changes but limits construct validity with respect to runtime semantic properties (control-plane convergence, routing dynamics, packet-level reachability under transient state). If operational claims require guarantees about runtime consequences, future studies should integrate emulation or simulation stages that exercise control-plane and dataplane effects and record dynamic assertions (e.g., convergence time, transient reachability). The archived manifest documents that our results are strictly conditional on the validator’s static semantics.

Design recommendations grounded in observed artifacts. Based on the run artifacts and contingency counts, we recommend three concrete design levers for future confirmatory NetOps evaluations: 1) Calibrate task difficulty or perform adversarial augmentation to target a nontrivial baseline failure prevalence or a predetermined discordant-pair count aligned with the planned absolute/relative effect size. Use deterministic task generators with adjustable difficulty knobs and preregister the target discordant count. 2) Explicitly preregister pairing semantics and repair budgets and incorporate their expected effects on discordant counts into power calculations; if the operational question aims to evaluate remediation of a single plan, prefer shared-candidate pairing but then set task difficulty to produce more initial failures. 3) When runtime semantics matter, add an emulation/simulation validation stage and preregister how emulator outcomes map to success/failure to avoid post-hoc redefinition of the estimand. All of these recommendations are traceable to artifacts in the execution manifest (task difficulty fields, validator violation codes, and the paired contingency table) and are therefore actionable.

Threats to validity revisited with artifact examples. Internal validity: the `shared_candidate` design and deterministic replacement rules are preregistered but reduce the number of informative discordant pairs; this is evident in the archived `stage_counts` and `provider_call` audit. Construct validity: static validator checks may miss dynamic failure modes; the validator traces explicitly show which checks were applied and which dynamic behaviors were not exercised. External valid-

ity: the synthetic task bank and constrained vendor templates are enumerated in the task manifest and limit generalization across vendor heterogeneity and production topologies. Conclusion validity: the paucity of discordant pairs (one) creates high sampling uncertainty for large preregistered effects; the paired bootstrap and exact McNemar results in the archived analysis artifacts document this limitation quantitatively.

Operational implications for deploying guarded pipelines. The run shows that a frozen deterministic validator plus a bounded repair loop can correct at least some invalid candidates without human intervention (the single observed correction). However, whether such a pipeline materially reduces first-pass misconfiguration in production depends on the baseline error prevalence and the validator’s coverage of real failure modes. When baseline error is already rare, the primary operational value of a validator+repair may be as a safety net that prevents the rare catastrophic mistake rather than as a mechanism to substantially increase throughput. Organizations should therefore align evaluation goals—error-reduction vs catastrophic-risk mitigation—with study design choices (task bank selection, pairing semantics, emulation inclusion) and with the operational acceptance thresholds that govern whether automated repair may be applied without human approval.

Reproducibility and archival practice. This run demonstrates the utility of a full artifact archive: per-task inputs, normalized responses, validator traces, repair prompts/responses, the paired contingency table, and the analysis code together enable independent recomputation of reported counts and intervals. The execution manifest documents `model_calls_used = 301` and `repair_stage` invocations = 1; these accounting numbers are minimal but sufficient to audit the reported outcomes. Future confirmatory NetOps work should adopt similar archival granularity to permit external verification and subsequent meta-analysis.

VI. LIMITATIONS

- Synthetic task bank and constrained vendor-template scope limit external validity to broad production environments.
- Single LLM (gpt-5-mini) and a single validator implementation restrict generality across model families and validator designs.
- The preregistered power plan assumed a baseline pass rate = 0.40; the observed baseline (299/300) contradicts that assumption and removed headroom to detect the preregistered $\geq 50\%$ relative reduction.
- Sampling semantics (shared_initial_candidate) reused the same initial candidate across paired arms, increasing within-pair correlation and reducing discordant pairs available to McNemar inference.
- Validation used deterministic configuration/constraint checks rather than full dynamic emulation of runtime semantic effects; actual runtime consequences of configurations were not executed and remain unmeasured.
- H2 repair-success evidence is based on a single initially invalid case (1/1 $\rightarrow$ 100%), yielding a wide Clopper–

Pearson 95% CI = [0.025, 1.0]; larger counts of initial failures are required to estimate repair effectiveness precisely.

VII. CONCLUSION

We executed a preregistered paired experiment (N=300 pairs) comparing single-shot LLM generation (gpt-5-mini) to a guarded pipeline consisting of a frozen deterministic validator and a single automated repair iteration. Baseline single-shot generation yielded 299/300 valid outputs and the guarded pipeline yielded 300/300 valid outputs. The single automated repair corrected the lone invalid baseline output, but the observed near-ceiling baseline contradicted the preregistered planning assumption (baseline pass rate = 0.40) and removed statistical headroom to detect the preregistered $\geq 50\%$ relative reduction. Future confirmatory NetOps evaluations should (a) calibrate task difficulty to produce sufficient initial failures or target discordant counts, (b) preregister pairing semantics and repair budgets explicitly, and (c) include emulation to measure runtime semantic effects when operational deployment claims are intended. All raw outputs, validator traces, repair traces, the execution manifest, and analysis artifacts are archived and enumerated in the Disclosure Statement to permit reproduction of the reported counts and intervals.

DISCLOSURE STATEMENT

This study was executed autonomously after the autonomy lock under the archived master prompt (provenance/master_prompt.txt; SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the master prompt prior to the autonomy lock; no manual human editing of manuscript technical content, figures, tables, or references occurred after the lock. Preregistration: cand-2026-001-A-prereg-v1 (the preregistration record was created before observing final outcomes; preregistration SHA-256 is recorded in the execution manifest). Declared model and tooling: gpt-5-mini (declared_model_version), adapter family hosted_netops_gvr_v1, frozen deterministic validator/repair adapter deterministic_netops_validator_v5. Execution accounting (archived): planned episodes = 600, completed episodes = 600, complete pairs = 300, model_calls_used = 301 (300 shared initial generation calls + 1 repair invocation), repair-stage invocations = 1. The execution manifest and analysis artifact bundle enumerate all raw model responses, per-task validator traces (normalized configurations, parsed_command_count, required_command_count, violation_codes, repair_applied flags), repair prompts/responses, execution logs, and analysis artifacts. The archived execution manifest and analysis bundle are the authoritative source for the numeric counts reported in this manuscript. The autonomous pipeline performed literature search, preregistration, execution, analysis, AI-peer-review style checks, and manuscript drafting autonomously after the autonomy lock in accordance with the CNSM Agentic AI

Researcher track requirements. The preregistered planning assumption used in power calculations (planned baseline pass rate = 0.40) is explicitly reported here because it materially affects interpretation of the confirmatory result.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Oghenekeno Hilkiyah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1
- [3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518
- [4] Goutam Adwant, Manjari Srivastav, "Evidence Coverage Evaluator: A Comprehensive Framework for Assessing Grounding Quality in Retrieval-Augmented Generation Systems", 2026, 10.36227/techrxiv.176784505.56675782/v1
- [5] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [6] Apostol Vassilev, Alina Oprea, Alie Jean Fordyce, Hyrum S. Anderson, "Adversarial machine learning :", 2024, 10.6028/nist.ai.100-2e2023